

Time Series Subsequence Matching using OpenMP and CUDA

**Performance optimization and scaling analysis on
Fault Detection Dataset**

Jacopo Bruscaaglioni

Introduction

Problem Definition & Motivation

The Problem: Subsequence similarity search in continuous multi-variate/long time series (e.g., fault detection in industrial sensor signals).

Algorithmic Complexity:

- Given N series of length L , and Q queries of length M , computing the Minimum Sum of Absolute Differences (SAD / L_1 norm) requires testing all sliding window positions: $O(Q \cdot N \cdot (L - M) \cdot M)$.

The Computational Bottleneck: As L and Q grow, sequential search takes hundreds of seconds, making real-time querying impossible without parallel acceleration.

In this work, performance benchmarking is carried out on the **FaultDetectionA** dataset. This real-world dataset was collected from an electromechanical drive system designed to monitor the operational condition of rolling bearings and detect structural damages. Specifically, we evaluate the test set, which comprises **2,728 univariate time series**, with each individual series consisting of **5,120 sensor readings**.



Introduction

Algorithmic Pipeline

- **Sequential Baseline:** Outer loops scan series and **Sliding Window offsets**; inner loop evaluates distance vector differences.
- **Hotspot Identification:** Most of runtime is spent *inside the distance accumulation kernel*:
 - `current_sad += | T[i+j] - Q[j] |` → executed for $Q \times N \times (L - M + 1) \times M$ times
 - $T[i + j]$ is the value of the j -th element of the sliding window with offset i and $Q[j]$ the j -th value of the Query
- **Target Platforms:**
 - **Multi-core CPU via OpenMP** (*Thread parallelism + SIMD vectorization*).
 - **Massively parallel GPU via CUDA** (*Two-dimensional grid decomposition + shared memory*).
- **Workload Scale:** For typical parameters ($N=2728$, $L=5120$, $Q=50$, $M=500$), this reaches **~315 billion operations**, necessitating parallel hardware acceleration.
- **Key Algorithmic Property:** Fully independent evaluation across query-series pairs and sliding offsets, enabling lock-free parallelism and high cache/shared-memory reuse.



CPU Parallelization

OpenMP, AoS vs SoA & SIMD

Memory Layout Comparison:

- **Array of Structures (AoS):** `std::vector<TimeSeries>` where each object owns separate heap allocations (`std::vector<double>`), causing cache misses and pointer chasing.
- **Structure of Arrays (SoA):** Flat contiguous memory buffer (`all_data_flat`), maximizing spatial locality and cache-line utilization.

Loop Scheduling & Vectorization:

- `#pragma omp parallel for collapse(2)` across queries and series to eliminate load imbalance.
- `#pragma omp simd reduction(+:current_sad)` to leverage hardware AVX/FMA vector registers.



CPU Parallelization

OpenMP, AoS vs SoA & SIMD

1. SAD_Search Kernel (SIMD)

```
int SAD_Search(const double* T, int
N,
const vector<double>& Q){
// T: 5120 misurazioni per query Q
double min_sad =
std::numeric_limits<double>::max();
int best_index = -1;
int M = Q.size(); // lunghezza query
const double* Q_ptr = Q.data();
for (int i=0; i<= N-M; ++i){
double current_sad = 0.0;
// SIMD + reduction
#pragma omp simd
reduction(+:current_sad)
for (int j=0; j<M; ++j){
current_sad += std::abs(T[i+j] -
Q_ptr[j]);
}
if (current_sad < min_sad) {
min_sad = current_sad;
best_index = i;
}
}
return best_index;
}
```

2. SoA: Contiguous Memory

```
#pragma omp parallel for collapse(2)
// SoA: Contiguous memory access
for (int q_idx = 0; q_idx <
numQueries; ++q_idx) {
for (int i = 0; i < numRows;
++i) {
const double* series_ptr =
&dataset.all_data_flat[i * L];
all_results[q_idx][i] =
SAD_Search(series_ptr, L,
queries[q_idx]);
}
}
```

3. AoS: Non-Contiguous Access

```
#pragma omp parallel for collapse(2)
// AoS: Non-contiguous access
for (int q_idx = 0; q_idx <
numQueries; ++q_idx) {
for (int i = 0; i < numRows;
++i) {
all_results[q_idx][i] =
SAD_Search(dataset[i].data.data(),
dataset[i].data.size(),
queries[q_idx]);
}
}
return all_results;
```

GPU Acceleration

CUDA Kernel Architecture

- **2D Grid Mapping:**
 - Grid: `dim3 blocksPerGrid(num_series, num_queries)` → each CUDA block is responsible for matching one query against one full time series.
- **Shared Memory & Memory Hierarchy:**
 - Fast cooperative staging of the entire time series into `__shared__ double s_mem[]` (eliminating repetitive global VRAM accesses).
 - Loop unrolling (`#pragma unroll 4`) for register-level instruction pipelining.
- **Parallel Tree Reduction:**
 - Inter-thread logarithmic stride reduction (`blockDim.x / 2`) within shared memory to locate the minimum distance and offset without thread divergence.



GPU Acceleration

CUDA Kernel Architecture

1. Memory Staging & Distance Calc

```
__global__ void search_kernel_SoA(
    const double* __restrict__ d_data,
    const double* __restrict__ d_queries,
    int num_series, int series_len,
    int query_len, int* d_results)
{
    const int series_idx = blockIdx.x;
    const int query_idx = blockIdx.y; // Indice query
    if (series_idx < num_series) {
        extern __shared__ double s_mem[];
        double* s_series = s_mem;
        size_t series_bytes = series_len * sizeof(double);
        size_t aligned_offset_bytes = (series_bytes + 15) & ~15;
        ReductionItem* s_red = (ReductionItem*)(s_mem +
            aligned_offset_bytes / sizeof(double));
        const int series_offset = series_idx * series_len;

        // Accesso SoA in Memoria Condivisa
        for (int t = threadIdx.x; t < series_len; t += blockDim.x) {
            s_series[t] = d_data[series_offset + t];
        }
        __syncthreads();

        // CALCOLO SLIDING WINDOW
        const int query_offset = query_idx * query_len;
        const int max_start = series_len - query_len;
        double min_distance = DBL_MAX;
        int best_match_idx = -1;
        for (int i = threadIdx.x; i <= max_start; i += blockDim.x) {
            double distance = 0.0;
            #pragma unroll 4
            for (int j = 0; j < query_len; ++j) {
                double diff = s_series[i + j] - d_queries[query_offset + j];
                distance += fabs(diff); // L1 distance
            }
            if (distance < min_distance) {
                min_distance = distance;
                best_match_idx = i;
            }
        }
    }
}
```

2. Tree Reduction & Output

```
// Salvataggio minimi locali per thread
s_red[threadIdx.x].dist = min_distance;
s_red[threadIdx.x].idx = best_match_idx;
__syncthreads();

// RIDUZIONE VISTA ALBERO (Shared Memory)
for (int stride = blockDim.x / 2; stride > 0; stride /= 2)
{
    if (threadIdx.x < stride) {
        if (s_red[threadIdx.x + stride].dist <
            s_red[threadIdx.x].dist) {
            s_red[threadIdx.x] = s_red[threadIdx.x + stride];
        }
    }
    __syncthreads();
}

// Thread 0 scrive il risultato finale in VRAM
if (threadIdx.x == 0) {
    int out_idx = query_idx * num_series + series_idx;
    d_results[out_idx] = s_red[0].idx;
}
}
```

Validation & Experimental Methodology

Validation:

- CPU vs GPU automated validation comparing indices and verifying equivalent minimums under tolerance (10^{-4}) for floating-point precision and duplicate matchings.

Rigorous Benchmarking:

- Discarded warm-up runs to mitigate cold caches and driver latency.
- Measured **mean**, **std dev**, **min**, and **max** across multiple runs (5 times) isolating computation time from disk I/O.

System Specs: CPU model, physical/logical core count, GPU model (SMs, VRAM), compiler flags (**-O3 -fopenmp**).

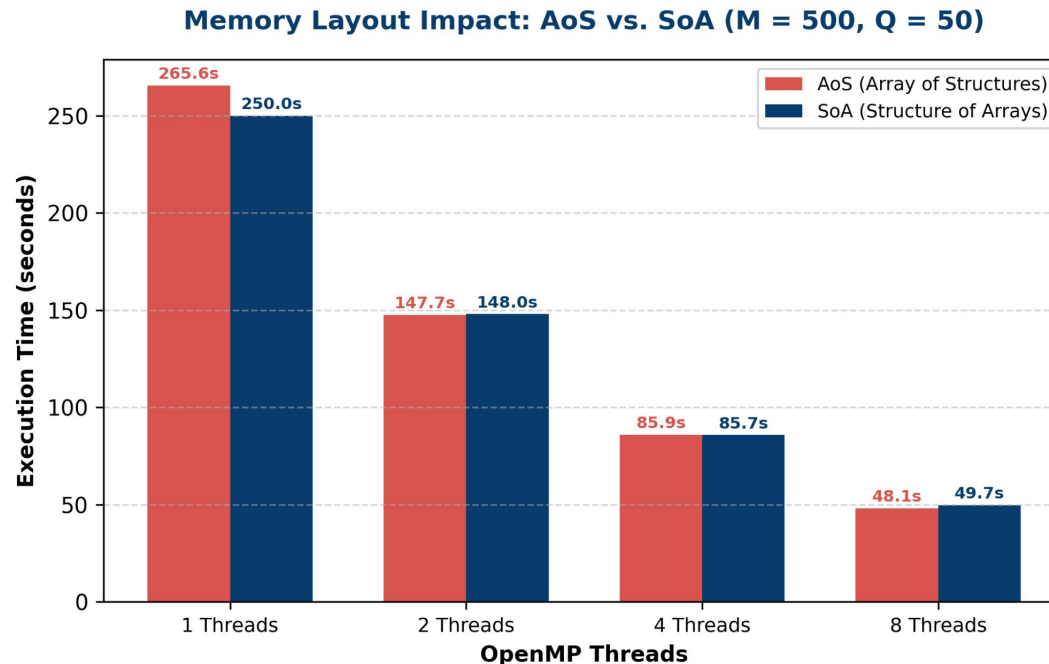
Components:

- **CPU Host (OpenMP Benchmarks):** Intel Core i7-8550U @ 1.80 GHz (4 Physical Cores / 8 Logical Threads, 8 MB Intel Smart Cache), compiled with **GCC/Clang -O3 -fopenmp -mavx2**.
- **GPU Cloud Platform (CUDA Benchmarks - Google Colab):** NVIDIA Tesla T4 (Turing Architecture, 40 SMs, 2,560 CUDA Cores, 16 GB GDDR6 @ 320 GB/s bandwidth), compiled with **nvcc -O3**.



Experimental Results

OpenMP Memory Layout Impact

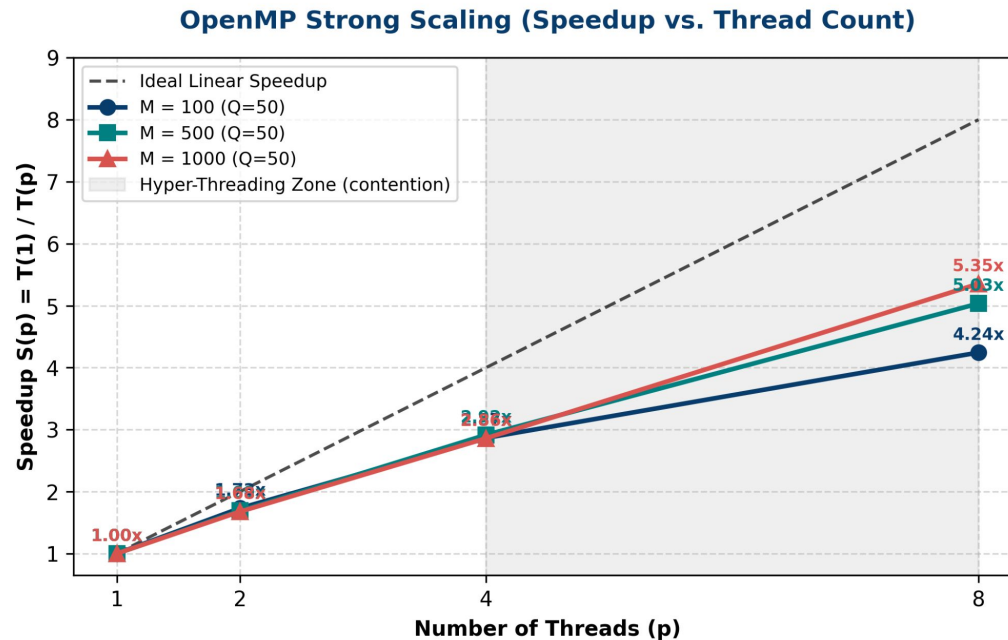


AoS vs SoA Comparison:

- **Single-Thread Cache Benefit:** SoA saves 15.6s (250.0s vs 265.6s) at 1 thread, achieving a 5.9% speedup from contiguous memory layout.
- **Vectorization Enabler:** Flat 1D layout (`all_data_flat`) eliminates pointer chasing and ensures stride-1 cache-line loads for `#pragma omp simd`.
- **Multi-Thread Convergence:** Across 2 to 8 threads, runtimes equalize (~85.8s at 4T, ~48.9s at 8T) as memory-bus saturation becomes the dominant bottleneck over cache-line alignment.

Experimental Results

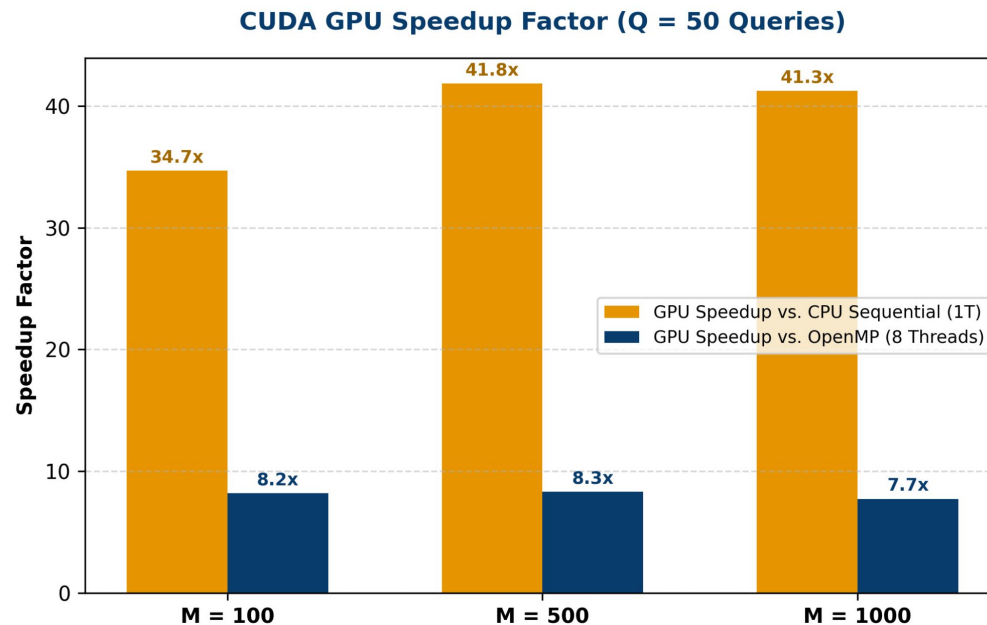
OpenMP Scalability



- **Scaling Trajectory:** Steady near-linear speedup up to **4 physical cores** (~2.9× at p=4).
- **Hyper-Threading Contribution (p=8):** Reaches up to **5.35× speedup** for M=1000, yielding an additional **1.87× gain** beyond physical cores via SMT latency hiding.
- **Workload Sensitivity:** Higher query lengths (M=1000) demonstrate better thread efficiency (5.35× vs 4.24×) due to increased compute-to-synchronization ratio.

Experimental Results

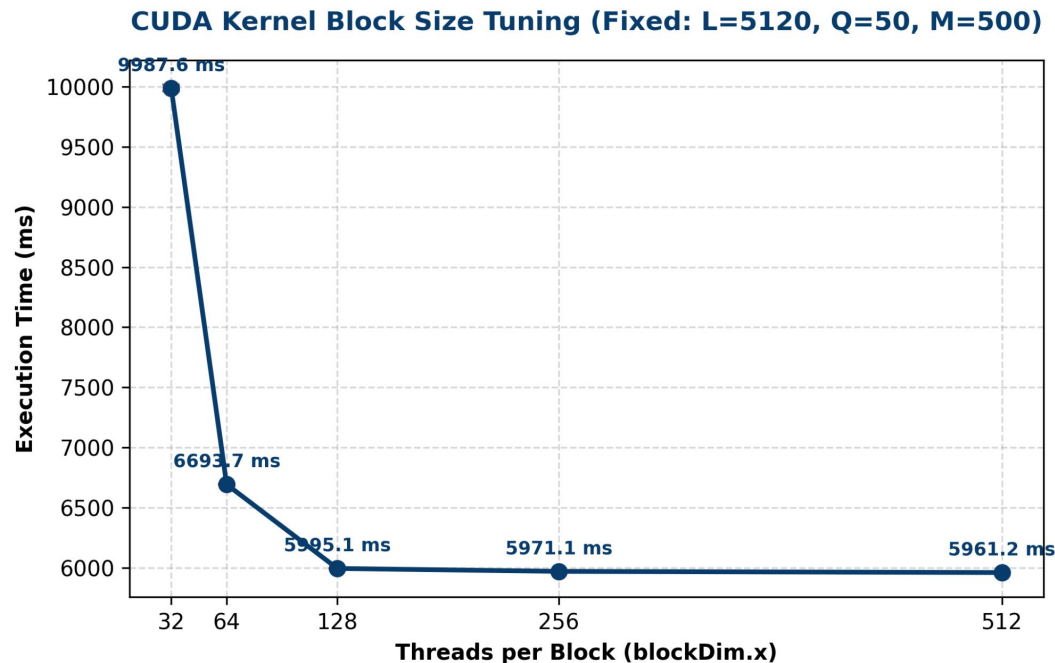
CUDA GPU Speedup Factor



- **Massive Baseline Speedup:** CUDA achieves a **34.7x to 41.8x speedup** over single-threaded sequential CPU execution.
- **Sustained Multi-Core Acceleration:** Yields an additional **7.7x to 8.3x speedup** directly over fully parallel 8-thread OpenMP.
- **Peak Compute Efficiency:** Maximum acceleration occurs at **M=500 (41.8x vs Seq / 8.3x vs OpenMP)**, representing the sweet spot between thread workload and cache tiling

Experimental Results

CUDA Empirical Block-Size Tuning

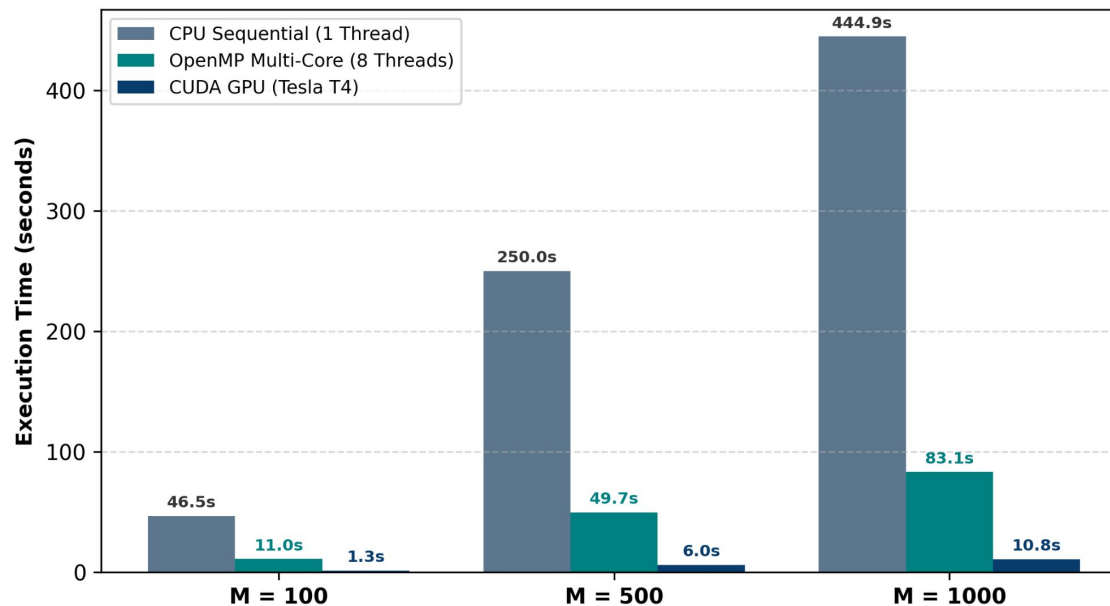


- **Underutilization at Low BlockDim:** Execution takes **9,987.6 ms at 32 threads** due to limited warp occupancy and pipeline stalls.
- **Occupancy Knee (128 threads):** Sharp **40% runtime drop** between 32 and 128 threads (5,995.1 ms), saturating warp scheduler queues.
- **Plateau (256–512 threads):** Optimal configuration reached at **512 threads (5,961.2 ms)**, maximizing SM occupancy while balancing shared memory allocation.

Global Performance Comparison

CPU vs OpenMP vs CUDA

Cross-Platform Execution Time (Batch Q = 50 Queries)



- **Sequential Bottleneck:** Sequential CPU scales rapidly with query size, exceeding **444.9 seconds** (~7.4 min) at M=1000.
- **OpenMP:** 8-thread multi-core execution reduces runtime down to **83.1 seconds** for M=1000 and **49.7 seconds** for M=500.
- **CUDA Real-Time Capability:** Tesla T4 slashes runtime to just **1.3s (M=100)** and **10.8s (M=1000)**, making interactive batch querying feasible.

Results Table

OpenMP N. Queries = 50 , Queries Length = 1000

Platform / Method	Memory Layout	Threads / Units	Mean Time (s)	Std Dev (s)	Speedup (vs. Baseline)	Speedup (vs. OMP 8T)
OpenMP	AoS	2 Threads	264.26 s	± 1.10 s	1.68×	0.31×
OpenMP	SoA	2 Threads	265.39 s	± 1.69 s	1.68×	0.31×
OpenMP	AoS	4 Threads	155.48 s	± 0.28 s	2.86×	0.53×
OpenMP	SoA	4 Threads	155.55 s	± 0.37 s	2.86×	0.53×
OpenMP	AoS	8 Threads	83.37 s	± 0.84 s	5.34×	1.00×
OpenMP	SoA	8 Threads	83.11 s	± 0.57 s	5.35×	1.00× (Ref OMP)



Results Table

CUDA Thread Block Size Tuning (N. Query = 50 , Query Length = 500)

BlockDim	Mean Time (ms)	Std Dev (ms)	Min Time (ms)	Max Time (ms)
32	9,987.63	± 27.53	9,966.10	10,024.70
64	6,693.72	± 0.12	6,693.57	6,693.88
128	5,995.08	± 0.16	5,994.89	5,995.30
256	5,971.11	± 0.49	5,970.64	5,971.72
512	5,961.20	± 0.63	5,960.25	5,961.74



Results Table

CPU vs OpenMP vs CUDA Benchmark Scaling

Query Length (M)	Num Queries (Q)	CPU Seq Time [1T] (ms)	OpenMP Time [8T] (ms)	CUDA Time (ms)	Speedup vs CPU Seq	Speedup vs OpenMP 8T
100	5	4,539.0	1,103.7	135.09	33.60×	8.17×
100	50	46,542.50	10,967.65	1,341.35	34.70×	8.18×
500	5	25,540.75	4,951.34	598.05	42.71×	8.28×
500	50	250,016.50	49,664.70	5,976.92	41.83×	8.31×
1000	5	45,241.00	8,603.11	1,059.17	42.71×	8.12×
1000	50	444,877.00	83,108.80	10,783.90	41.25×	7.71×



Conclusions & Key Findings

Core Achievements

Algorithmic Acceleration

Eliminated computational bottleneck on FaultDetectionA dataset.

- CUDA achieved **33.6×–42.7×** speedup over baseline.
- **7.7×–8.3×** direct speedup vs 8-thread OpenMP.

Memory Exploitation

- **AoS to SoA:** Stride-1 contiguous access enabled hardware SIMD.
- **Shared Memory:** Bypassed high-latency global VRAM reads.

Empirical Tuning CUDA

128–512 threads/block yielded a **40.3% runtime reduction** against the 32 threads/block configuration.

Best Configuration

Platform

NVIDIA Tesla T4 (Kernel: search_kernel_SoA)

Data Layout

SoA 1D flattened buffer (all_data_flat)

Grid & Block Geometry

2D Grid (num_series, num_queries) with 128–512 threads/block.

Throughput Performance

50 queries across 2,728 series (>315B ops):

1.3 s (M=100)

10.8 s (M=1000)

Transformed multi-minute batch queries into interactive real-time processing.

Limitations & Bottlenecks

CPU Scalability Cap

OpenMP saturates beyond 4 physical cores due to shared L3 cache contention and memory bus limits.

GPU Memory Transfers

PCIe Host-to-Device transfer creates initial overhead; requires batching to keep dataset VRAM-resident.

Shared Memory Limits

Allocating target series in `__shared__` double limits max series length **L** without sub-tiling.